

序列扫描与区间状态

怎样让边界移动、摘要复用和候选淘汰都有证明

408 · 数据结构 Algorithm Extension A01

母模型：已确认区、候选区与永久淘汰

序列技巧的共同链条是

Order / Coordinate Contract → Active Interval or Result Prefix → Incremental Summary
→ Dominance / Boundary Proof → One-Pass Cost.

指针移动只有在被跳过的元素以后不可能重新成为答案时才安全；摘要只有在能从旧摘要和新端点更新时才值得维护。模板名称不是正确性证明。

Owner 与停止边界

- **Owns**：前缀/差分、窗口、快慢/左右指针、矩阵边界和候选支配。
- **Uses**：DS01 顺序/矩阵表示、DS02 单调队列、DS09 有序索引、DS10 Hash 计数。
- **Stops**：二分答案的单调谓词归 DS09；链表指针改边归 DS01；单调栈/队列容器不变量归 DS02。

目录

1	前缀和与差分：把重复区间工作变成端点事件	2
1.1	前缀值频次：把“区间等式”改写成“历史查询”	2
1.2	左右摘要并不要求可逆：除自身以外的乘积	2
1.3	差分的密集数组与稀疏事件	3
2	滑动窗口：可恢复的活动区间	3
2.1	最长合法窗口与最短覆盖窗口	3
3	快慢指针：结果前缀与扫描后缀	3
4	左右指针：单调性与支配淘汰	4
4.1	有序两数和与三数和	4
4.2	容器与接雨水	4
5	矩阵：坐标变换与四边界收缩	4
6	题型路由：先识别可复用信息，再选模板	5
7	复杂度与反例验证	6
8	压缩复原	6

1 前缀和与差分：把重复区间工作变成端点事件

对数组 $a_0, \dots, a_{n-1}$ ，定义半开前缀

$$P[0] = 0, \quad P[i+1] = P[i] + a_i.$$

闭区间 $[l, r]$ 的和为 $P[r+1] - P[l]$ 。 $n+1$ 个位置的哨兵使 $l=0$ 、空区间和边界统一，不需要为第一项写特殊分支。二维前缀把一维差分扩展为容斥：

$$P[i+1][j+1] = a_{ij} + P[i][j+1] + P[i+1][j] - P[i][j].$$

查询矩形时以四个前缀加减，左上角被减两次所以加回。

差分是前缀的逆操作：

$$D[0] = a_0, \quad D[i] = a_i - a_{i-1}.$$

对闭区间 $[l, r]$ 加 v 只改两个端点： $D[l] += v$ ，若 $r+1 < n$ 则 $D[r+1] -= v$ ；最后做一次前缀恢复。 m 次修改加恢复的成本是 $O(m+n)$ ，但修改过程中不能直接读取最终数组，除非另有在线结构。

半开事件避免端点混乱

把乘客在 $from$ 上车、在 to 下车表示为事件 $+num$ 于 $from$ 、 $-num$ 于 to ，扫描站点时先累加事件再检查容量。这里区间是 $[from, to)$ ；若题意规定在终点之后才离开，则应改成 $to+1$ 。端点语义属于问题合同，不能死记某一道题的下标。

前缀技巧依赖所用聚合可逆：加法可以用相减恢复，异或可用再次异或恢复；一般的 ‘min’/‘max’ 没有这样的逆运算，不能机械套两个前缀相减。溢出、负数、空数组和二维矩阵的行列方向都必须进入边界检查。

1.1 前缀值频次：把“区间等式”改写成“历史查询”

若目标是统计和为 k 的连续子数组，令扫描到当前位置后的前缀为 s 。任何以当前位置结尾的合法区间都对应一个历史前缀 $s-k$ ，所以维护

$$\text{freq}[x] = \text{此前前缀值 } x \text{ 出现的次数}.$$

处理当前元素时必须先把 $\text{freq}[s-k]$ 加入答案，再把当前 s 写入频次；反过来会在 $k=0$ 时把“当前前缀减自身”的空区间错误计入。初始化 $\text{freq}[0]=1$ 表示数组开始前的空前缀，使从下标 0 开始的合法区间无需特判。该方法允许负数，因为它不依赖窗口和的单调性，期望时间 $O(n)$ 、空间 $O(n)$ ，其中 Hash 的期望成本前提归 DS10。

同一个改写还生成两类变体。若要找最长区间，Hash 保存某前缀值的**最早位置**而不是出现次数，且第一次出现后不覆盖；若二元数组要求 0 与 1 数量相等，可先把 0 映射为 -1 ，问题变成“和为零的最长区间”。统计数量、求最长长度和判断存在性使用的是不同摘要，不能只因都叫“前缀 + Hash”就共享更新合同。

1.2 左右摘要并不要求可逆：除自身以外的乘积

有些题不做任意区间查询，而是为每个位置组合“左侧全部信息”和“右侧全部信息”。除自身以外数组乘积可令输出先保存左侧乘积，再从右向左用一个滚动后缀乘积相乘，时间 $O(n)$ 、除输出外空间 $O(1)$ ，且无需除法，因此自然覆盖零元素。它不是两个前缀相除：乘法遇到零不可逆。这里可复用的是**两侧摘要分解**，不是前缀和公式。

1.3 差分的密集数组与稀疏事件

当坐标范围小而连续，可用长度固定的差分数组承载事件。全零初始数组无需先做相邻差；航班编号从 1 开始时，闭区间 $[first, last]$ 应映射为 $D[first-1] += v$ 、若 $last < n$ 则 $D[last] -= v$ 。拼车则是半开行程 $[from, to)$ ，在 to 直接减。两个公式的差别来自业务端点，不来自题名。

当坐标很大但事件很少，不应为最大坐标分配整段数组；可把“坐标 $\rightarrow$ 净变化”存入有序事件表，按坐标排序后扫描。会议室并发数、日程最大重叠等问题都可由事件前缀生成，但同一坐标的开始/结束先后必须由半开或闭区间合同决定。若任务要求在线区间修改与在线查询，离线差分已经越过停止边界，应转向 Fenwick/线段树等动态结构，而不是在每次查询前都 $O(n)$ 恢复。

2 滑动窗口：可恢复的活动区间

窗口维护 $[left, right)$ 和一个随端点增删的摘要。典型框架是右端加入元素，若窗口违反约束则左端不断移出，窗口重新合法后更新答案。正确性依赖一个单调性条件：在右端固定时，若某个左端已使窗口合法/非法，向某一方向移动左端不会出现不可预测的多次来回。若约束不满足该条件，双指针可能漏解。

2.1 最长合法窗口与最短覆盖窗口

无重复最长子串维护字符计数；右端加入导致计数超过 1 时，左端移除直到恢复合法，每个字符至多进出一次，所以总时间 $O(n)$ 。最小覆盖窗口的状态不只是计数，还要维护“已满足的种类数”或缺口数；只有当所有需求满足时才收缩并记录最短答案。固定长度排列窗口则在每次移动时增删一个字符，不能把“满足后继续收缩”误用于固定 k 。

可变窗口的完整状态协议是 `need/window/valid`：`need[c]` 表示目标需求，`window[c]` 表示当前计数，只有当某字符计数首次达到需求时才增加 `valid`，从达到状态跌破时才减少。最小覆盖在窗口已经覆盖全部需求时，于移除左端之前记录候选；无重复最长子串在恢复合法后记录最大长度。更新时机若互换，前者会错过即将被破坏的最短合法窗，后者会把仍含重复的窗口算入答案。

固定长度排列/异位词窗口保持 $right - left = |T|$ ：右端每加入一项，就在超长时恰好移出左端，再判断计数是否完全匹配。它不需要寻找“最短”，也不允许在合法后任意继续缩小。窗口最大值虽然长度也固定，但摘要要是 DS02 Own 的单调队列；A01 只负责索引过期条件 $index < left$ 和端点推进。

什么时候不能用普通窗口

“和为 k 的最长子数组”在有负数时不具备按右端扩张、左端收缩的单调性；窗口会错过解，应改用前缀和 + Hash 或其他状态。窗口模板适合的是可由端点单调修复的约束，而不是所有子数组题。

滑动窗口最大值需要 DS02 的单调队列：窗口端点负责过期，队尾弹出被新值支配的索引，队头才是最大值。队列实现若使用线性 `'shift()'`，代码层面的总复杂度会被破坏，应使用头指针或真正的双端队列。

3 快慢指针：结果前缀与扫描后缀

原地筛选数组时，‘fast’ 扫描未处理后缀，‘slow’ 指向结果区的下一个写入位置。循环不变量是： $[0, slow)$ 已经是按题意保留的结果， $[fast, n)$ 尚未读取；若当前值满足保留条件，就写入 `'nums[slow]'` 并推进，否则只推进 ‘fast’。因此删除元素、去重和移动零都能在 $O(n)$ 、 $O(1)$ 辅助空间完成。

覆盖与交换不是同一语义：覆盖只承诺前缀有效，尾部可以是垃圾；移动零若要求保留非零元素相对顺序，可以用交换或稳定覆盖后补零，必须在合同中声明。

若有序数组允许每个值最多保留 q 次，写指针为 $slow$ 时，只需在 $slow < q$ 或当前值不等于结果前缀中倒数第 q 个值时写入：

$$\text{keep}(x) \iff slow < q \vee x \neq a[slow - q].$$

这个式子把“去重一次”和“最多保留两次”统一为结果前缀约束；比较对象必须来自已经确认的输出区，而不是未处理原数组。含退格字符串的逆向扫描则维护待跳过字符数：遇退格增加计数，遇普通字符且计数非零就消耗一次跳过；两串各找下一个有效字符后比较。它仍是“扫描后缀逐步提交结果”的边界模型，但写入方向和状态不同。

链表快慢指针属于 DS01 的具体表示应用。本册只抽取“速度比导致相遇/中点”的推理：快指针每次两步、慢指针一步，若有环则相对速度使它们在有限圈内相遇；相遇后一个指针回到头部、两个同速前进，利用距离等式得到环入口。空链、单节点和‘n’超长时的先后指针必须由 dummy 与边界条件覆盖。

4 左右指针：单调性与支配淘汰

左右指针从两端向内收缩，只有在某一侧移动后能永久排除一批候选时才正确。

4.1 有序两数和与三数和

有序数组中若 $a_l + a_r < \text{target}$ ，固定 r 时任何更小的左值都不可能达到目标，因此可以安全地增加 l ；若和过大则对称减少 r 。三数和先排序并固定一个元素，把剩余问题降为左右指针；外层和内层都必须跳过相邻重复值，避免输出重复而不误删不同位置的合法组合。

k 数之和可以递归固定一个值，把 k 降为 $k - 1$ ，直到两数和；每层只跳过同一递归深度的相邻重复候选，命中一组两数和后同时越过左右两侧的重叠值。若把“值相等”不分层级地全局跳过，会删掉需要多个相同数且索引不同的合法答案。最接近目标的三数和则不在命中后收集所有组合，而是维护当前绝对误差最小值，并仍用和的单调方向移动端点。排序成本 $O(n \log n)$ ，三数主体 $O(n^2)$ ；固定更高 k 后成本按递归层数增长，输出规模还需单独报告。

4.2 容器与接雨水

容器面积为 $(r - l) \min(h_l, h_r)$ 。宽度每步都减少，若左边较短，移动右边不能提高当前瓶颈高度且宽度变小，故右端在这一轮被支配；只能移动短边寻找更高瓶颈。接雨水维护左右最大高度，若 ' $\text{leftMax} < \text{rightMax}$ '，左侧水量已由 ' leftMax ' 决定，右侧未知不会改变它，因此结算左端并右移；等号时采用固定的一侧即可，但要保持不重计。

5 矩阵：坐标变换与四边界收缩

顺时针旋转正方形可分解为主对角线转置再逐行翻转；两步都是原地交换，组合后每个坐标 (i, j) 映射到 $(j, n - 1 - i)$ 。螺旋遍历维护 ' $\text{top}, \text{bottom}, \text{left}, \text{right}$ ' 四个边界，每走完上、右、下、左边就收缩对应边界；走下边和左边前必须再次检查边界，才能避免单行/单列重复。

由坐标映射可复原三种旋转，而不必分别背代码：

变换	(i, j) 的新位置	一种原地分解
90° 顺时针	$(j, n - 1 - i)$	主对角线转置 + 逐行翻转
90° 逆时针	$(n - 1 - j, i)$	主对角线转置 + 逐列翻转
180°	$(n - 1 - i, n - 1 - j)$	逐行翻转 + 逐列翻转

这些原地分解要求方阵；一般 $m \times n$ 矩阵旋转后形状变成 $n \times m$ ，若存储不能重解释形状，就需要新容器。

螺旋读取与螺旋生成共享同一 **frontier**：前者把边界元素追加到结果，后者把递增值写入边界。每条边完成后立即收缩，因此在只剩一行或一列时，下边/左边必须重新检查合法性。对角线遍历则用 $i - j$ 标识主对角线平行族、用 $i + j$ 标识副对角线平行族；若输出方向交替，还需由对角线编号奇偶决定反转，而不是改变分组不变量。

矩阵边界算法的循环不变量是：边界外已输出/已写入，边界内尚未处理；每一轮至少收缩一条边，故每个格子最多访问一次。对角线编号由 $i - j$ 或 $i + j$ 不变量给出；矩阵置零、生命游戏等原地题还需要额外编码“旧状态”和“新状态”，不能把已经修改的值误当作邻居旧值。

矩阵原地修改的两阶段提交

若新状态依赖周围旧状态，扫描阶段必须同时保留 **old/new** 两层语义：可以使用额外位、枚举过渡态或第一行/列作为标记。只有全部单元都完成读取后，第二遍才把过渡态提交为最终状态。直接边读边覆盖会让后续邻居读到混合时间层。

6 题型路由：先识别可复用信息，再选模板

题面工作负载	首选状态	停止/转交信号
静态多次区间聚合	前缀摘要	聚合不可逆或需要在线修改
离线多次区间增量	端点差分事件	稀疏坐标改事件表；在线查询改动态树
连续区间且约束可单 调修复	滑动窗口	负数等使收缩方向失去单调性
原地稳定筛选/压缩	结果前缀 + 扫描后缀	尾部也有语义时补写/交换合同
有序候选的配对/支 配	左右指针	无序且不能排序，或移动侧无法证明淘汰
二维边界访问/坐标 置换	四边界或坐标映射	新状态依赖旧邻居时采用两阶段提交
单调可行答案空间	DS09 二分谓词	判定函数不单调

7 复杂度与反例验证

模式	典型成本	必须证明的事
前缀/差分	构建/恢复 $O(n)$ ，查询或修改 $O(1)$	哨兵和端点语义
窗口	双端点总移动 $O(n)$	收缩方向不会漏解
快慢筛选	$O(n)$ 、 $O(1)$ 辅助空间	结果前缀不变量
左右指针	常见 $O(n)$ 或排序后 $O(n^2)$	移动侧的支配淘汰
矩阵边界	$O(mn)$	边界外完成、单行单列不重复

验证时至少攻击：全相等、严格递增、严格递减、空/单元素、重复端点、负数窗口、奇偶长度、单行/单列矩阵和超过整数范围。若改变一个前提后模板失效，应把前提写进 **Stop Boundary**，而不是继续套用。

8 压缩复原

忘记代码时间：当前已确认区是什么？活动边界采用闭区间还是半开区间？摘要加入/移出如何更新？被丢弃元素凭什么永不成为答案？窗口约束是否单调？矩阵每圈收缩哪条边？答案必须能复原循环不变量、端点公式和复杂度。

一句话压缩

序列算法的核心不是“两根指针”，而是维护一个可解释的边界与摘要，并证明每次移动都扩大已确认区域或永久淘汰候选；没有单调性、可逆性或支配关系时，应停止套模板并换用 DP、Hash、排序或二分等 Owner。